

Entrega 2 Sistemas Operacionais e Computação em Nuvem

Nesta entrega eu utilizei uma instância EC2 na AWS para executar três scripts Python baseados em três aulas da matéria de Inteligência Artificial e Machine Learning. Utilizei também o pacote tmux do terminal Linux para exibir na parte inferior do terminal as métricas constantemente da instância, o tmux é como se fosse um “gerenciador de sessões do terminal”, ele permite que algo seja exibido em um canto do terminal sem atrapalhar outras operações de serem realizadas, isto porque ele gerencia várias sessões na mesma janela. Neste caso ele exibe o resultado do meu script de monitoramento e permite que eu realize o treinamento dos modelos enquanto isso. Eu utilizei um dataset do Github para treinar três tipos de modelos de ML básicos: classificação, regressão linear e regressão logística.

Utilizei uma instância EC2 na AWS do tipo **t3.large** para realizar os treinamentos dos modelos.

Para os modelos de regressão linear e regressão logística eu tive que fazer algumas alterações pois nas aulas de IA e ML utilizamos também a biblioteca matplotlib para visualizar através de gráficos os resultados dos treinamentos, isto não é possível através do terminal Linux, nas aulas do professor Victor nós utilizamos o Colab para executar estes códigos.

Código do Script de monitoramento

Este é um script que monitora as métricas da instância do **EC2** e mostra visualmente na parte inferior da tela através de um pacote chamado **tmux** que serve para gerenciar janelas dentro do próprio terminal Linux, permitindo visualizar constantemente as

métricas recolhidas pelo script enquanto o usuário utiliza o terminal normalmente sem interrupções.

```
#!/bin/bash
# EC2 User Data 2 - Amazon Linux
# Instala Python e configura monitor de CPU/RAM/Disco/Rede/Processos via tmux status bar
# NOTA: Para aplicar manualmente, use os comandos com "tee" no final deste arquivo.

set -e

# Atualizar sistema e instalar dependências
yum update -y
yum install -y python3 python3-pip tmux procps-ng bc sysstat

# Criar script de monitoramento
cat > /usr/local/bin/sys-monitor.sh << 'EOF'
#!/bin/bash
bar() {
    local pct=$1
    local filled=$((pct / 20))
    local empty=$((5 - filled))
    local b=""
    for ((i=0; i<filled; i++)); do b+="█"; done
    for ((i=0; i<empty; i++)); do b+="░"; done
    echo "$b"
}

cpu_idle=$(top -bn1 | grep "Cpu(s)" | awk '{print $8}')
cpu_used=$(echo "100 - ${cpu_idle:-0}" | bc 2>/dev/null || echo "0")
cpu_int=${cpu_used%.*}
mem_info=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
disk_info=$(df / | awk 'NR==2 {gsub("%",""); print $5}')
procs=$(ps aux --no-heading | wc -l)
rx=$(cat /proc/net/dev | awk '/eth0|ens/ {gsub(/:/, ""); printf "%.1f", $2/1024/1024}')
tx=$(cat /proc/net/dev | awk '/eth0|ens/ {gsub(/:/, ""); printf "%.1f", $10/1024/1024}')
echo "CPU[${bar $cpu_int}]${cpu_int}% RAM[${bar $mem_info}]${mem_info}% DSK[${bar $disk_info}]${disk_info}%
P:${procs} NET:${rx}/${tx}MB"
EOF

chmod +x /usr/local/bin/sys-monitor.sh

# Configurar tmux status bar
cat > /etc/tmux.conf << 'EOF'
set -g status-interval 5
set -g status-right-length 100
set -g status-right '#(sys-monitor.sh)'
set -g status-style 'bg=black,fg=green'
```

```

set -g status-right-style 'fg=cyan'
EOF

# Auto-start tmux em sessões interativas
cat > /etc/profile.d/tmux-auto.sh << 'EOF'
if command -v tmux &>/dev/null && [ -z "$TMUX" ] && [[ $- == *i* ]]; then
    tmux new-session -A -s main
fi
EOF

chmod +x /etc/profile.d/tmux-auto.sh

```

Código do modelo de classificação

```

# Pacotes Les Charger
from sklearn.ensemble import VotingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix
import pandas as pd

# Leia o arquivo
df = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv")

# Mostre as 5 linhas do conjunto de dados
df.head()

df.describe()# Resumo Estatístico

df.info()# DataSet de informação

df.isnull().sum()

# As variáveis explicativas ou as características de cada indivíduo são representadas pelos oito primeiros atributos

```

```

# Conjunto de dados e a variável a ser prevista é o resultado.

# Crie uma variável de dados que contém apenas recursos
data = df.loc[:, df.columns != 'Outcome']

# Armazenar dados de destino na variável de destino

target = df['Outcome']

# Divisão de dados
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.3, random_state = 0)

# Crie árvore com uma instância da classe Classify da árvore de decisão
tree = DecisionTreeClassifier(max_depth = 5)
# Treine o algoritmo (use ajuste)
tree.fit(X_train, y_train)
# Calcule a escore F.
y_pred = tree.predict(X_test)
print(f"Decision Tree - F1 Score: {f1_score(y_test, y_pred):.4f}")

# Crie KNN com uma instância da classe KNeighborsClassifier
knn = KNeighborsClassifier(metric='manhattan', n_neighbors = 7)
# Preserive o algoritmo (ajuste)
knn.fit(X_train, y_train)

# Calcule a escore F.
y_pred = knn.predict(X_test)
print(f"KNN - F1 Score: {f1_score(y_test, y_pred):.4f}")

# Crie NB, uma instância de classe com os parâmetros padrão GaussianNB
nb = GaussianNB()
# Treine o algoritmo (ajuste USAR)
nb.fit(X_train, y_train)

# Calcule a escore F.
y_pred = nb.predict(X_test)
print(f"Naive Bayes - F1 Score: {f1_score(y_test, y_pred):.4f}")

# Cálculo da medida F (escore F1)
vc = VotingClassifier(estimators=[('tree', tree), ('knn', knn), ('nb', nb)], voting='hard')
vc.fit(X_train, y_train)

```

```
# Avaliação do modelo
y_pred = vc.predict(X_test)
print(f"Voting Classifier - F1 Score: {f1_score(y_test, y_pred):.4f}")
```

Código do modelo de regressão linear

```
# Regressão Linear - Dataset Diabetes
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import pandas as pd
import numpy as np

# Leia o arquivo
df = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv")

# Para regressão linear, prever o valor de Glucose com base nos outros atributos
data = df.loc[:, ~df.columns.isin(['Glucose', 'Outcome'])]
target = df['Glucose']

# Divisão de dados
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.3, random_state=0)

# Criar e treinar o modelo de Regressão Linear
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predição
y_pred = lr.predict(X_test)

# Métricas
print("=== Regressão Linear ===")
print(f"R² Score: {r2_score(y_test, y_pred):.4f}")
print(f"MAE (Erro Absoluto Médio): {mean_absolute_error(y_test, y_pred):.4f}")
print(f"MSE (Erro Quadrático Médio): {mean_squared_error(y_test, y_pred):.4f}")
print(f"RMSE (Raiz do Erro Quadrático Médio): {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
print(f"\nCoefficientes:")
for col, coef in zip(data.columns, lr.coef_):
    print(f" {col}: {coef:.4f}")
print(f"Intercepto: {lr.intercept_:.4f}")
```

Código do modelo de regressão logística

```
# Regressão Logística - Dataset Diabetes
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, accuracy_score, precision_score, recall_score, confusion_matrix
import pandas as pd

# Leia o arquivo
df = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv")

# Variáveis explicativas e variável alvo
data = df.loc[:, df.columns != 'Outcome']
target = df['Outcome']

# Divisão de dados
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.3, random_state=0)

# Criar e treinar o modelo de Regressão Logística
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

# Predição
y_pred = lr.predict(X_test)

# Métricas
print("=== Regressão Logística ===")
print(f"Acurácia: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precisão: {precision_score(y_test, y_pred):.4f}")
print(f"Recall: {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.4f}")
print(f"\nMatriz de Confusão:")
print(confusion_matrix(y_test, y_pred))
```

Instalando dependências:

```
us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-0cc63b5d12d613751?region=us-east-2#:  
Getting Started it.amazon.com Amazon Internal Tr... Seu plano de integ... Reward Centre Amazon Meetings (M)Wiki Nossos clientes | E... Isengard | pheanto... Concur  
Session ID: pheanto-isengard-zgpcjz4lgyjybbztryzjdtzeGi Shortcuts Instance ID: i-0cc63b5d12d613751 (Test_Server) Terminate session  
Package procs-ng-3.3.17-1.amzn2023.0.2.x86_64 is already installed.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
Installing: python3-pip	noarch	21.3.1-2.amzn2023.0.17	amazonlinux	1.8 M
tmux	x86_64	3.2a-3.amzn2023.0.2	amazonlinux	478 k
Installing weak dependencies: libxcrypt-compat	x86_64	4.4.33-7.amzn2023	amazonlinux	92 k

```
=====
```

Transaction Summary			
Install 3 Packages			
Total download size: 2.3 M			
Installed size: 12 M			
Downloading Packages:			
(1/3): libxcrypt-compat-4.4.33-7.amzn2023.x86_64.rpm	1.8 MB/s	92 kB	00:00
(2/3): tmux-3.2a-3.amzn2023.0.2.x86_64.rpm	7.5 MB/s	478 kB	00:00
(3/3): python3-pip-21.3.1-2.amzn2023.0.17.noarch.rpm	23 MB/s	1.8 MB	00:00
Total	19 MB/s	2.3 MB	00:00

```
=====
```

Running transaction check			
Transaction check succeeded.			
Running transaction test			
Transaction test succeeded.			
Running transaction			
Preparing	:		1/1
Installing	:	libxcrypt-compat-4.4.33-7.amzn2023.x86_64	1/3
Installing	:	python3-pip-21.3.1-2.amzn2023.0.17.noarch	2/3
Installing	:	tmux-3.2a-3.amzn2023.0.2.x86_64	3/3
Running scriptlet:	:	tmux-3.2a-3.amzn2023.0.2.x86_64	3/3
Verifying	:	libxcrypt-compat-4.4.33-7.amzn2023.x86_64	1/3
Verifying	:	python3-pip-21.3.1-2.amzn2023.0.17.noarch	2/3
Verifying	:	tmux-3.2a-3.amzn2023.0.2.x86_64	3/3

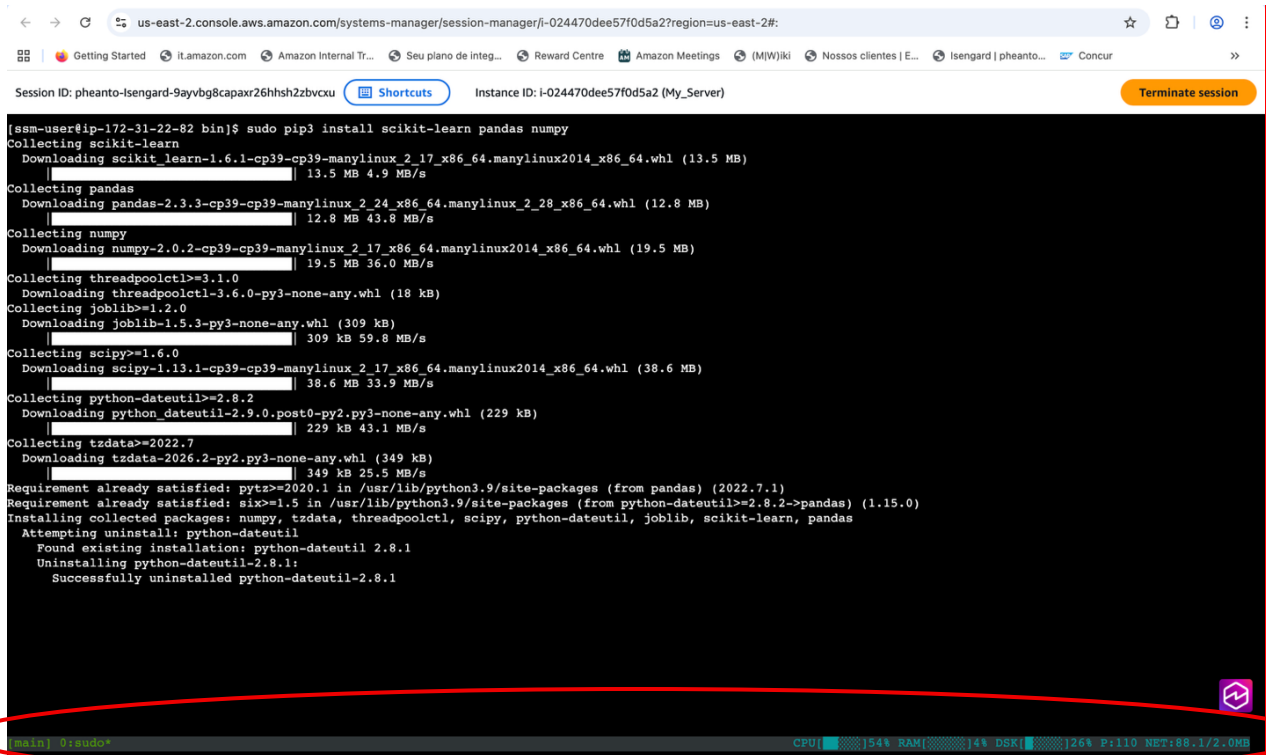
```
=====
```

Installed:			
libxcrypt-compat-4.4.33-7.amzn2023.x86_64	python3-pip-21.3.1-2.amzn2023.0.17.noarch	tmux-3.2a-3.amzn2023.0.2.x86_64	

```
Complete!  
sh-5.2$
```

Monitoramento

A partir deste ponto, sempre será exibido no canto inferior da tela através do tmux o resultado do meu script de monitoramento, onde é exibido o uso de RAM, CPU e disco, além do número de processos em execução e o uso de dados de rede.

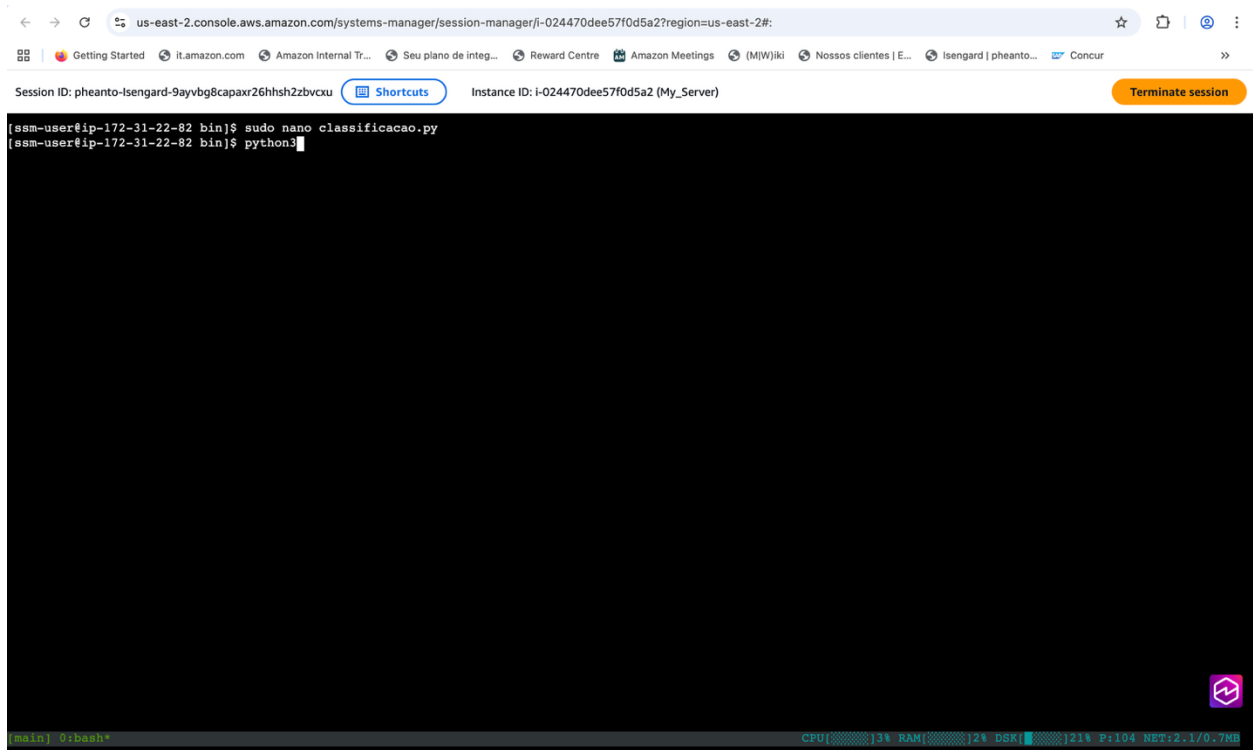


The screenshot shows a web browser window displaying the AWS Systems Manager console. The URL is `us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#`. The session ID is `pheanto-isengard-9ayvbg8capxr26hsh2zbtvcxu` and the instance ID is `i-024470dee57f0d5a2 (My_Server)`. A "Shortcuts" button and a "Terminate session" button are visible. The terminal window shows the execution of `sudo pip3 install scikit-learn pandas numpy`. The output lists the collection and download of various packages: `scikit-learn` (13.5 MB), `pandas` (12.8 MB), `numpy` (19.5 MB), `threadpoolctl` (18 kB), `joblib` (309 kB), `scipy` (38.6 MB), `python-dateutil` (229 kB), and `tzdata` (349 kB). It also shows the installation of these packages and the uninstallation of `python-dateutil-2.8.1`. At the bottom of the terminal window, a status bar displays system metrics: `[main] @raudo*` on the left and `CPU 54% RAM 4% DSK 26% P:110 NET:88.1/2.0MB` on the right. This status bar is circled in red.

```
[ssm-user@ip-172-31-22-82 bin]$ sudo pip3 install scikit-learn pandas numpy
Collecting scikit-learn
  Downloading scikit_learn-1.6.1-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.5 MB)
    | 13.5 MB 4.9 MB/s
Collecting pandas
  Downloading pandas-2.3.3-cp39-cp39-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.8 MB)
    | 12.8 MB 43.8 MB/s
Collecting numpy
  Downloading numpy-2.0.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (19.5 MB)
    | 19.5 MB 36.0 MB/s
Collecting threadpoolctl>=3.1.0
  Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Collecting joblib>=1.2.0
  Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
    | 309 kB 59.8 MB/s
Collecting scipy>=1.6.0
  Downloading scipy-1.13.1-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (38.6 MB)
    | 38.6 MB 33.9 MB/s
Collecting python-dateutil>=2.8.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
    | 229 kB 43.1 MB/s
Collecting tzdata>=2022.7
  Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
    | 349 kB 25.5 MB/s
Requirement already satisfied: pytz>=2020.1 in /usr/lib/python3.9/site-packages (from pandas) (2022.7.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil>=2.8.2->pandas) (1.15.0)
Installing collected packages: numpy, tzdata, threadpoolctl, scipy, python-dateutil, joblib, scikit-learn, pandas
  Attempting uninstall: python-dateutil
    Found existing installation: python-dateutil 2.8.1
    Uninstalling python-dateutil-2.8.1:
      Successfully uninstalled python-dateutil-2.8.1
[main] @raudo* CPU 54% RAM 4% DSK 26% P:110 NET:88.1/2.0MB
```

Como você pode ver, as métricas são exibidas dentro da área circulado em vermelho, no canto inferior da tela.

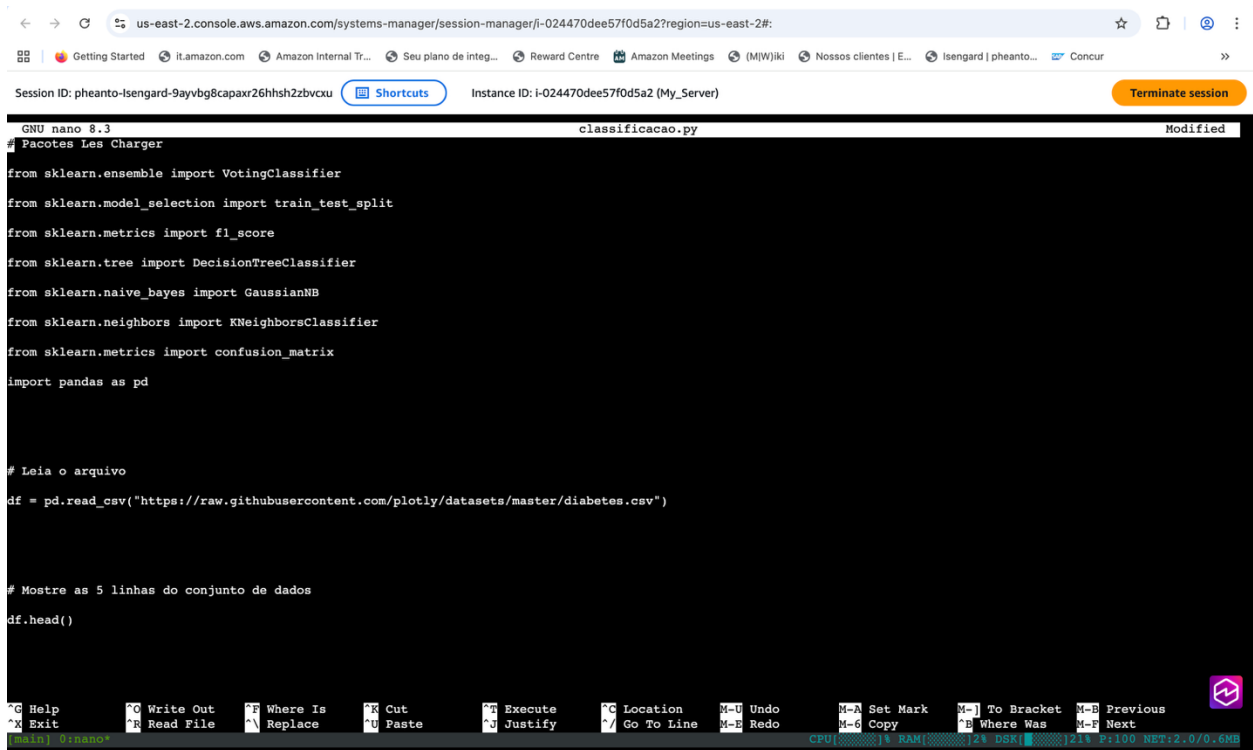
Escrevendo o arquivo de treinamento:



The screenshot shows the AWS Systems Manager console interface. At the top, the browser address bar displays the URL: `us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#:`. Below the address bar, there's a navigation bar with various links like 'Getting Started', 'it.amazon.com', 'Amazon Internal Tr...', 'Seu plano de integ...', 'Reward Centre', 'Amazon Meetings', '(M)Wiki', 'Nossos clientes | E...', 'Isengard | pheanto...', and 'Concur'. The main content area shows the session details: 'Session ID: pheanto-lsengard-9ayvbg8capaxr26hsh2zbvcxu' and 'Instance ID: i-024470dee57f0d5a2 (My_Server)'. A 'Shortcuts' button is visible next to the session ID. On the right, there's a 'Terminate session' button. The terminal window shows the following commands and output:

```
[ssm-user@ip-172-31-22-82 bin]$ sudo nano classificacao.py
[ssm-user@ip-172-31-22-82 bin]$ python3
```

The terminal status bar at the bottom indicates: `[main] 0:baah*` and system metrics: `CPU[ ] 3% RAM[ ] 2% DSK[ ] 21% P:104 NET:2.1/0.7MB`.



This screenshot shows the same AWS Systems Manager console interface, but the terminal window now displays the content of the file `classificacao.py` being edited in the nano editor. The file name 'classificacao.py' is shown in the top right corner of the editor window, along with the status 'Modified'. The code in the file is as follows:

```
GNU nano 8.3 classificacao.py
# Pacotes Les Charger

from sklearn.ensemble import VotingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix

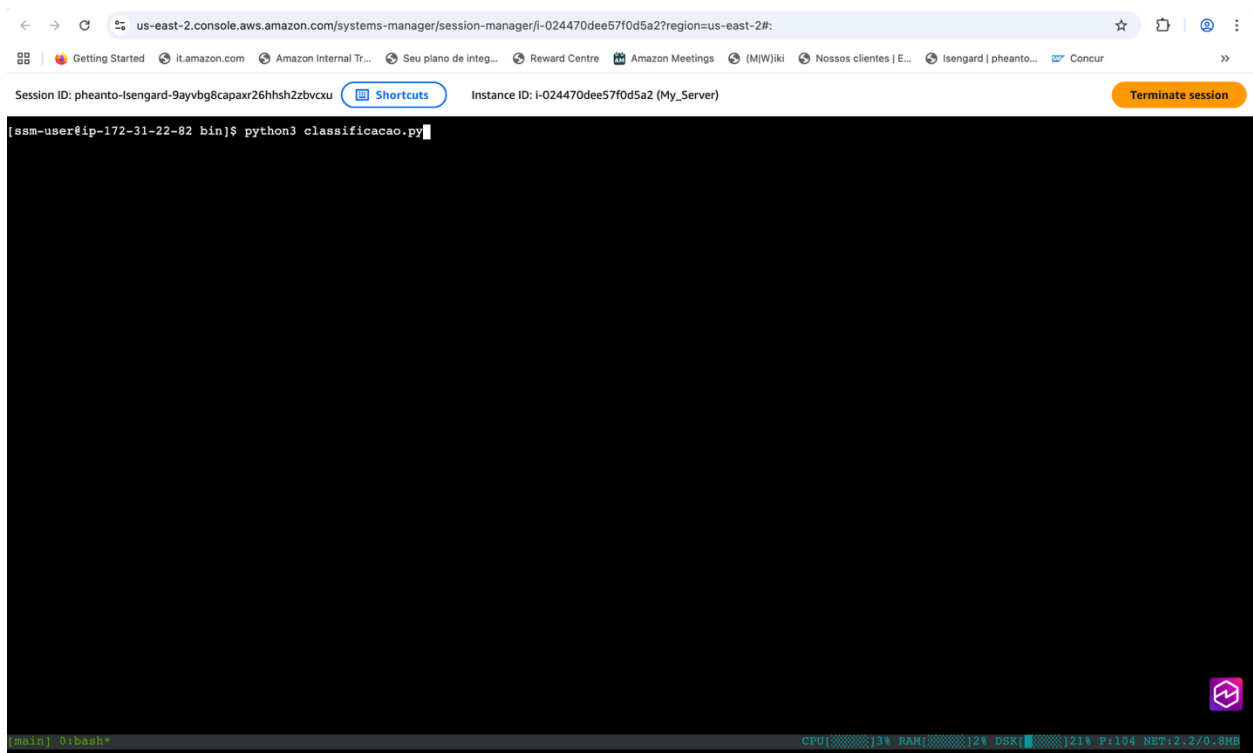
import pandas as pd

# Leia o arquivo
df = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv")

# Mostre as 5 linhas do conjunto de dados
df.head()
```

The terminal status bar at the bottom shows: `[main] 0:nano*` and system metrics: `CPU[ ] 1% RAM[ ] 2% DSK[ ] 21% P:100 NET:2.0/0.6MB`. A detailed nano editor help menu is visible at the bottom of the terminal window.

Executando o arquivo de Treinamento por classificação:



The screenshot displays the AWS Systems Manager console interface. At the top, the browser address bar shows the URL: `us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#:`. Below the address bar, a navigation bar contains links such as "Getting Started", "it.amazon.com", "Amazon Internal Tr...", "Seu plano de integ...", "Reward Centre", "Amazon Meetings", "(M)Wiki", "Nossos clientes | E...", "Isengard | pheanto...", and "Concur".

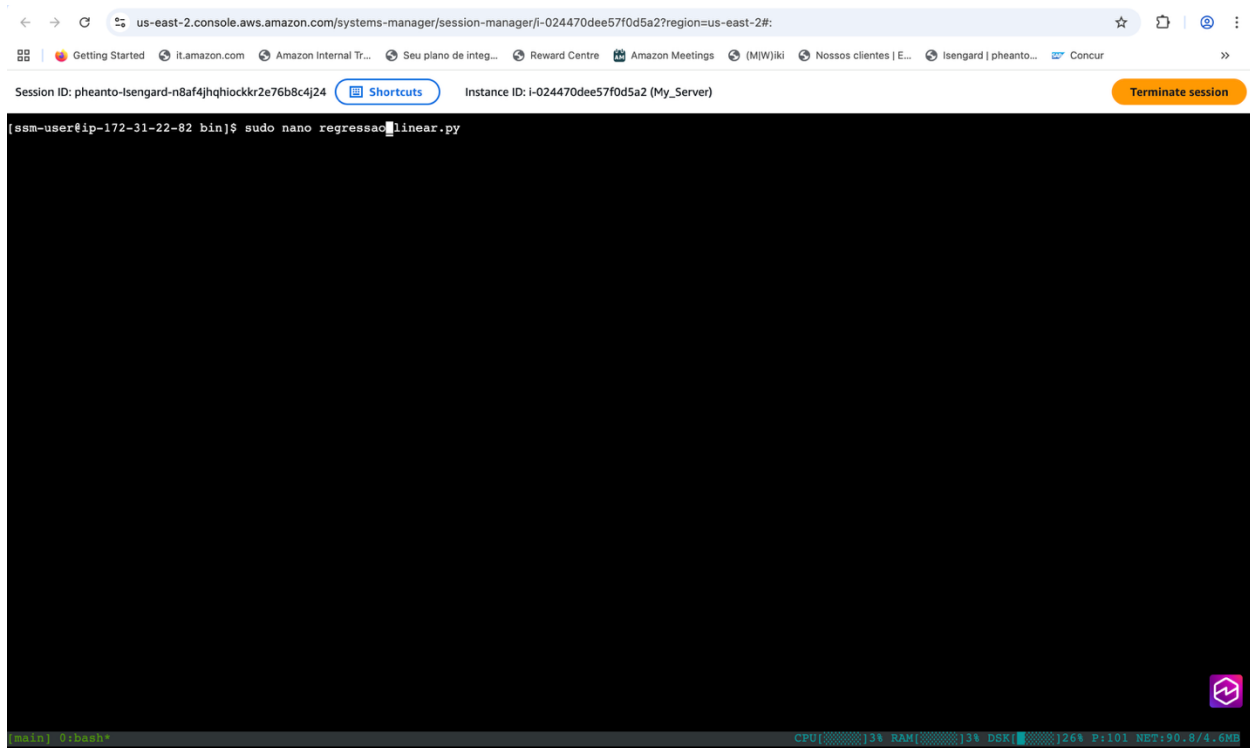
The main content area shows the session details for a specific instance. The "Session ID" is `pheanto-isengard-9ayvb8capaxr26hsh2zbvcxu`, and the "Instance ID" is `i-024470dee57f0d5a2 (My_Server)`. A "Shortcuts" button is visible next to the Session ID. On the right side, there is a yellow "Terminate session" button.

The terminal window shows the command `python3 classificacao.py` being executed. The terminal output is mostly black, indicating that the script is running. At the bottom of the terminal, a status bar displays system metrics: `CPU: 13% RAM: 12% DSK: 121% P: 104 NET: 2.2/0.6MB`. The prompt `[main] 0:baah*` is visible on the left.

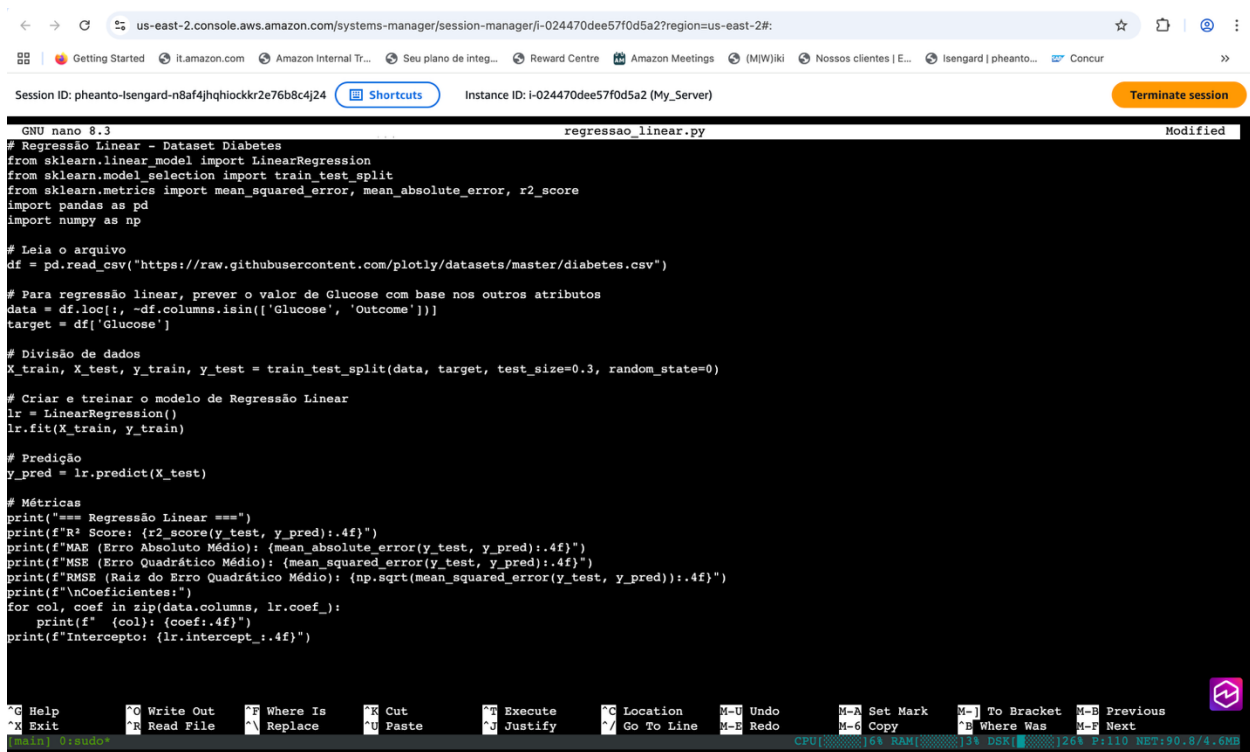
Resultado de diferentes métodos de classificação como KNN e árvore de decisão com seus respectivos valores F1 Score:

```
us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#:  
Getting Started it.amazon.com Amazon Internal Tr... Seu plano de integ... Reward Centre Amazon Meetings (M)Wiki Nossos clientes | E... Isengard | pheanto... Concur  
Session ID: pheanto-isengard-9ayvbg8capaxr26hsh2zbvcxu Shortcuts Instance ID: i-024470dee57f0d5a2 (My_Server) Terminate session  
[ssm-user@ip-172-31-22-82 bin]$ python3 classificacao.py  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 768 entries, 0 to 767  
Data columns (total 9 columns):  
# Column Non-Null Count Dtype  
---  
0 Pregnancies 768 non-null int64  
1 Glucose 768 non-null int64  
2 BloodPressure 768 non-null int64  
3 SkinThickness 768 non-null int64  
4 Insulin 768 non-null int64  
5 BMI 768 non-null float64  
6 DiabetesPedigreeFunction 768 non-null float64  
7 Age 768 non-null int64  
8 Outcome 768 non-null int64  
dtypes: float64(2), int64(7)  
memory usage: 54.1 KB  
Decision Tree - F1 Score: 0.5957  
KNN - F1 Score: 0.5547  
Naive Bayes - F1 Score: 0.5802  
Voting Classifier - F1 Score: 0.5909  
[ssm-user@ip-172-31-22-82 bin]$  
[main] 0ibash* CPU: 1% RAM: 3% DSK: 126% P: 106 NET: 90.2/4.0MB
```

Escrevendo o código de regressão linear:



The screenshot shows an AWS console session manager window. The browser address bar displays the URL: `us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#`. The session ID is `pheanto-lsengard-n8af4jhqhiocckr2e76b8c4j24` and the instance ID is `i-024470dee57f0d5a2 (My_Server)`. The terminal window shows the command `[ssm-user@ip-172-31-22-82 bin]$ sudo nano regressao_linear.py` being executed. The terminal is currently empty, and the nano editor interface is not yet visible.



The screenshot shows the same AWS console session manager window, but now the nano editor is open, displaying the contents of the file `regressao_linear.py`. The code is written in Python and uses the `sklearn` library for linear regression. The code includes imports for `LinearRegression`, `train_test_split`, `mean_squared_error`, `mean_absolute_error`, `r2_score`, `pandas`, and `numpy`. It reads a CSV file from a GitHub repository, splits the data into training and testing sets, fits a linear regression model, and prints various metrics including `R² Score`, `MAE`, `MSE`, and `RMSE`. The code also prints the coefficients and the intercept of the model.

```
GNU nano 8.3 regressao_linear.py Modified
# Regressão Linear - Dataset Diabetes
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import pandas as pd
import numpy as np

# Leia o arquivo
df = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv")

# Para regressão linear, prever o valor de Glucose com base nos outros atributos
data = df.loc[:, ~df.columns.isin(['Glucose', 'Outcome'])]
target = df['Glucose']

# Divisão de dados
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.3, random_state=0)

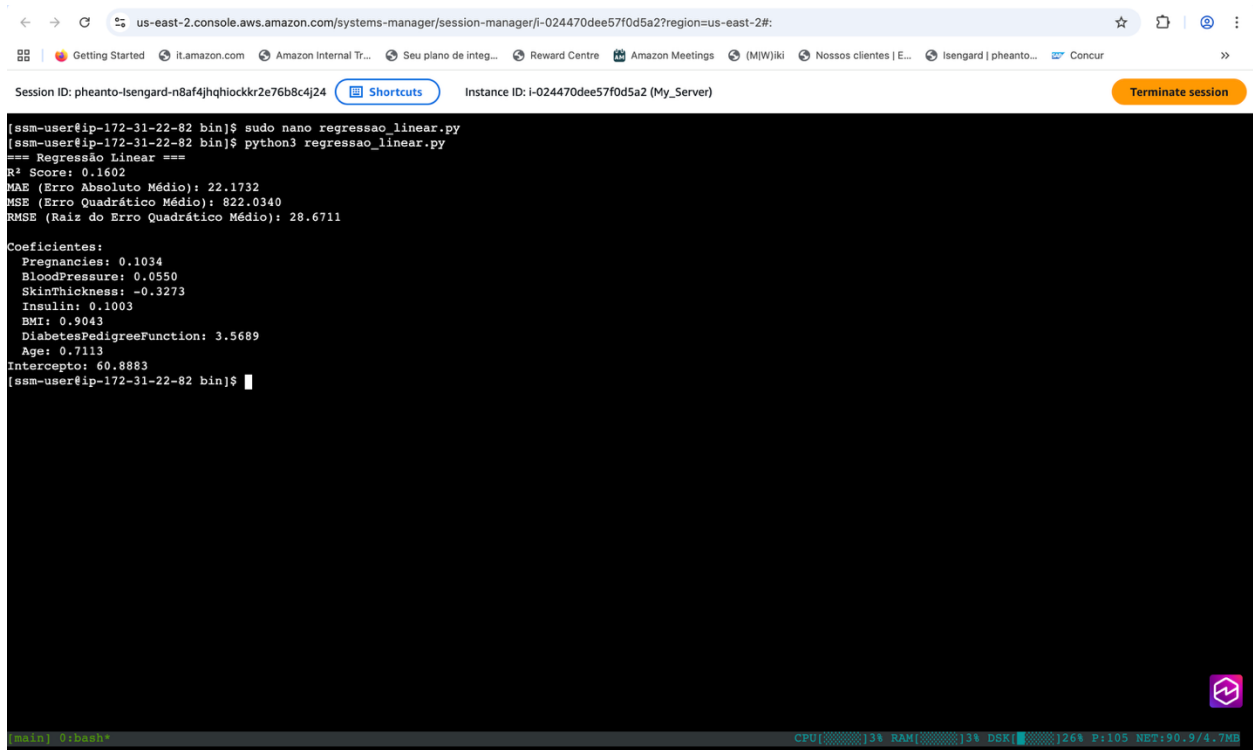
# Criar e treinar o modelo de Regressão Linear
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predição
y_pred = lr.predict(X_test)

# Métricas
print("=== Regressão Linear ===")
print(f"R² Score: {r2_score(y_test, y_pred):.4f}")
print(f"MAE (Erro Absoluto Médio): {mean_absolute_error(y_test, y_pred):.4f}")
print(f"MSE (Erro Quadrático Médio): {mean_squared_error(y_test, y_pred):.4f}")
print(f"RMSE (Raiz do Erro Quadrático Médio): {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
print(f"Coeficientes:")
for col, coef in zip(data.columns, lr.coef_):
    print(f"  {col}: {coef:.4f}")
print(f"Intercepto: {lr.intercept_:.4f}")

^G Help      ^O Write Out  ^F Where Is   ^X Cut        ^T Execute    ^C Location   ^U Undo       ^M Set Mark   ^J To Bracket ^P Previous
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^/_ Go To Line  ^M-E Redo     ^M-G Copy     ^B Where Was  ^M-F Next
[main] 0:sudo+
```

Resultado do modelo de regressão linear:



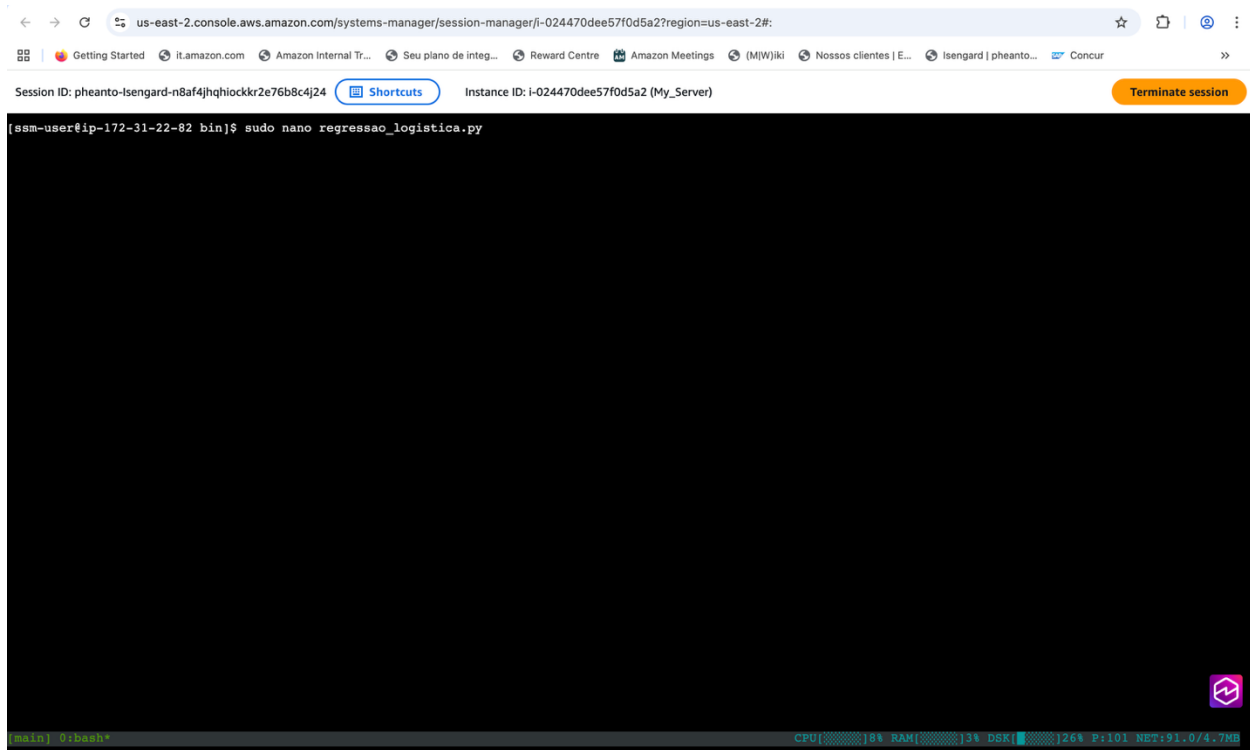
The screenshot shows a terminal window within an AWS Systems Manager session. The terminal displays the execution of a Python script named `regressao_linear.py`. The output includes the R-squared score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). It also lists the coefficients for various features and the intercept.

```
[ssm-user@ip-172-31-22-82 bin]$ sudo nano regressao_linear.py
[ssm-user@ip-172-31-22-82 bin]$ python3 regressao_linear.py
=== Regressão Linear ===
R² Score: 0.1602
MAE (Erro Absoluto Médio): 22.1732
MSE (Erro Quadrático Médio): 822.0340
RMSE (Raiz do Erro Quadrático Médio): 28.6711

Coeficientes:
Pregnancies: 0.1034
BloodPressure: 0.0550
SkinThickness: -0.3273
Insulin: 0.1003
BMI: 0.9043
DiabetesPedigreeFunction: 3.5689
Age: 0.7113
Intercepto: 60.8883
[ssm-user@ip-172-31-22-82 bin]$
```

At the bottom of the terminal window, a status bar shows system metrics: CPU usage at 13%, RAM at 13%, disk usage at 26%, and other system information.

E por fim escrevendo o código do modelo de regressão logística:



us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#:

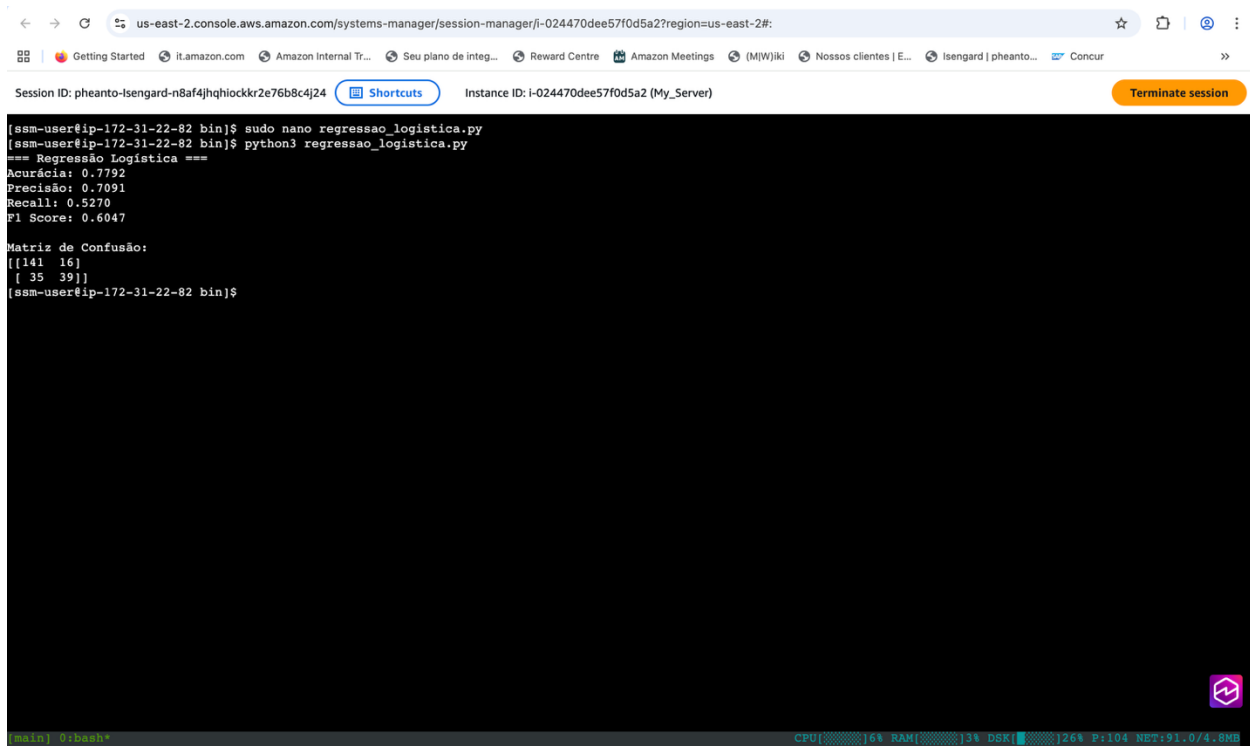
Getting Started it.amazon.com Amazon Internal Tr... Seu plano de integ... Reward Centre Amazon Meetings (M)Wiki Nossos clientes | E... Isengard | pheanto... Concur

Session ID: pheanto-lsengard-n8af4jhqiockkr2e76b8c4j24 Shortcuts Instance ID: i-024470dee57f0d5a2 (My_Server) Terminate session

```
[ssm-user@ip-172-31-22-82 bin]$ sudo nano regressao_logistica.py
```

[main] 0:hash* CPU:18% RAM:3% DSK:26% P:101 NET:91.0/4.7MB

O resultado da regressão logística junto com suas métricas foi:



us-east-2.console.aws.amazon.com/systems-manager/session-manager/i-024470dee57f0d5a2?region=us-east-2#:

Getting Started it.amazon.com Amazon Internal Tr... Seu plano de integ... Reward Centre Amazon Meetings (M)Wiki Nossos clientes | E... Isengard | pheanto... Concur

Session ID: pheanto-lsengard-n8af4jhqiockkr2e76b8c4j24 Shortcuts Instance ID: i-024470dee57f0d5a2 (My_Server) Terminate session

```
[ssm-user@ip-172-31-22-82 bin]$ sudo nano regressao_logistica.py
[ssm-user@ip-172-31-22-82 bin]$ python3 regressao_logistica.py
=== Regressão Logística ===
Acurácia: 0.7792
Precisão: 0.7091
Recall: 0.5270
F1 Score: 0.6047

Matriz de Confusão:
[[141 16]
 [ 35 39]]
[ssm-user@ip-172-31-22-82 bin]$
```

[main] 0:hash* CPU:16% RAM:3% DSK:26% P:104 NET:91.0/4.8MB

Para escrever o script de monitoramento eu utilizei o nano com o comando:

```
sudo nano /tmp/setup.sh
```

Para executar o script de monitoramento utilizei o seguinte comando:

```
sudo bash /tmp/setup.sh
```

Para iniciar o tmux manualmente:

```
tmux new-session -A -s main
```